Full Name : Ghrib Ala Eddine
Class : Second Year of Bachelor's in Informatics
Group : 5(A)
Module : OS

TP3 : Creating and Managing Processes under Linux

Part 01 : Simple Process Creation with "fork()"

```
alaghrib@alaghrib:~
alaghrib :: ~ » nano fork1.c
alaghrib :: ~ » gcc fork1.c -o fork1
alaghrib :: ~ » ./fork1
Je suis le processus père
PID père = 31466
Je suis le processus fils
PID fils = 31467
PPID fils = 31466
alaghrib :: ~ »
```

```
nano fork1.c
  GNU nano 7.2                                  fork1.c *
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Je suis le processus fils\n");
        printf("PID fils = %d\n", getpid());
        printf("PPID fils = %d\n", getppid());
    } else {
        printf("Je suis le processus père\n");
        printf("PID père = %d\n", getpid());
    }
    return 0;
}
```

Questions :
1 - 2 processes are created : the parent (PID=31466) and the child (PID=31467).
2 - fork() returns the child's PID, which is 31467.
3 - fork() returns 0.

4 - Because the parent and child run in parallel. The OS scheduler decides which one runs first — this is called scheduling non-determinism. In this execution, the parent printed first, then the child.
5 -

```
alaghrib :: ~ » pstree -p
systemd(1)─┬─ModemManager(900)─┬─{ModemManager}(913)
           │                   ├─{ModemManager}(914)
           │                   └─{ModemManager}(919)
           ├─NetworkManager(11052)─┬─{NetworkManager}(11053)
           │                       ├─{NetworkManager}(11054)
           │                       └─{NetworkManager}(11055)
           ├─accounts-daemon(751)─┬─{accounts-daemon}(781)
           │                      ├─{accounts-daemon}(782)
           │                      └─{accounts-daemon}(811)
           ├─agetty(1043)
           ├─at-spi2-registr(1357)─┬─{at-spi2-registr}(1363)
           │                       ├─{at-spi2-registr}(1364)
           │                       └─{at-spi2-registr}(1365)
           ├─avahi-daemon(11440)───avahi-daemon(11442)
           ├─blueman-tray(1674)─┬─{blueman-tray}(1698)
           │                    ├─{blueman-tray}(1699)
           │                    └─{blueman-tray}(1700)
           ├─bluetoothd(755)
           ├─boltd(902)─┬─{boltd}(909)
           │            ├─{boltd}(910)
           │            └─{boltd}(912)
           ├─colord(1423)─┬─{colord}(1438)
           │              ├─{colord}(1439)
           │              └─{colord}(1441)
           ├─crashhelper(2933)───{crashhelper}(2934)
           ├─cron(756)
           ├─csd-printer(1527)─┬─{csd-printer}(1528)
           │                   ├─{csd-printer}(1529)
           │                   └─{csd-printer}(1530)
           ├─cups-browsed(1802)─┬─{cups-browsed}(1813)
           │                    ├─{cups-browsed}(1814)
           │                    └─{cups-browsed}(1815)
           ├─cupsd(1005)
           ├─dbus-daemon(757)
           ├─fwupd(31266)─┬─{fwupd}(31267)
           │              ├─{fwupd}(31274)
           │              ├─{fwupd}(31275)
           │              ├─{fwupd}(31276)
           │              └─{fwupd}(31278)
           ├─ibus-daemon(1237)─┬─ibus-dconf(1255)─┬─{ibus-dconf}(1265)
           │                   │                  ├─{ibus-dconf}(1266)
           │                   │                  ├─{ibus-dconf}(1268)
           │                   │                  └─{ibus-dconf}(1277)
           │                   ├─ibus-engine-sim(1475)─┬─{ibus-engine-sim}(1477)
           │                   │                       ├─{ibus-engine-sim}(1478)
           │                   │                       └─{ibus-engine-sim}(1479)
           │                   ├─ibus-extension-(1258)─┬─{ibus-extension-}(1347)
           │                   │                       ├─{ibus-extension-}(1348)
           │                   │                       ├─{ibus-extension-}(1367)
           │                   │                       ├─{ibus-extension-}(1369)
           │                   │                       ├─{ibus-extension-}(1370)
           │                   │                       └─{ibus-extension-}(1427)
           │                   ├─ibus-ui-gtk3(1256)─┬─{ibus-ui-gtk3}(1337)
           │                   │                    ├─{ibus-ui-gtk3}(1338)
           │                   │                    ├─{ibus-ui-gtk3}(1350)
           │                   │                    ├─{ibus-ui-gtk3}(1351)
```

Part 02 : Synchronization with 'wait()'

```
alaghrib@alaghrib:~
alaghrib :: ~ » nano fork_wait.c
alaghrib :: ~ » gcc fork_wait.c -o fork_wait
alaghrib :: ~ » ./fork_wait
Fils : PID=37336
Père : PID=37335
```

Questions :

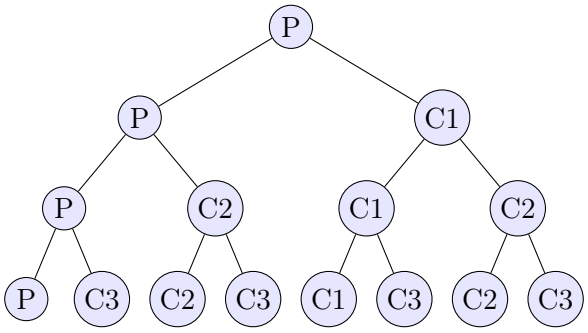
1 - the wait() function suspends the execution of the parent process until its child process terminates .

2 - The parent and child will run concurrently without synchronization, and the parent might finish before the child .

3 - Because wait() allows the parent to read the termination status of the child, allowing the system to fully remove the child from the process table.

Part 03 : Multiple Creation (Hierarchy)

Questions :
1 - A total of 8 processes are created (1 initial parent process and 7 child processes).
2 - The total number of processes is 2^n, where n is the number of fork() calls.
Since there are 3 fork() calls, the result is 2^3=8.
3 - The process draw :

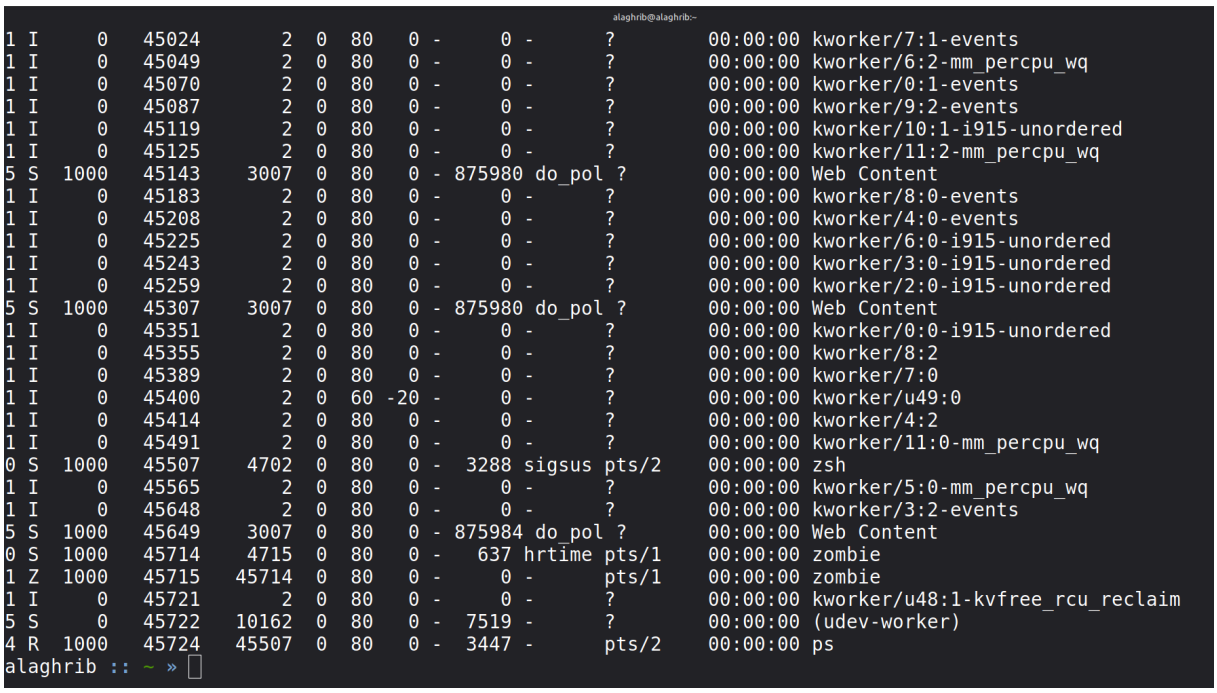


4 - Verification with 'pstree -p'

```
|-fork2(39627)-+-fork2(39628)-+-fork2(39630)---fork2(39634)
|              |              `-fork2(39633)
|              |-fork2(39629)---fork2(39632)
|              `-fork2(39631)
```

FIGURE 1 – I extract this from terminal

Part 04 : Zombie Process



Questions :

1 - Two processes are created : the original parent process and one child process generated by the fork() call .

2 - The child process finishes its execution and prints "Fils terminé", but because the parent is in a sleep() state and has not called wait(), the child remains in the system's process table as a "Zombie".

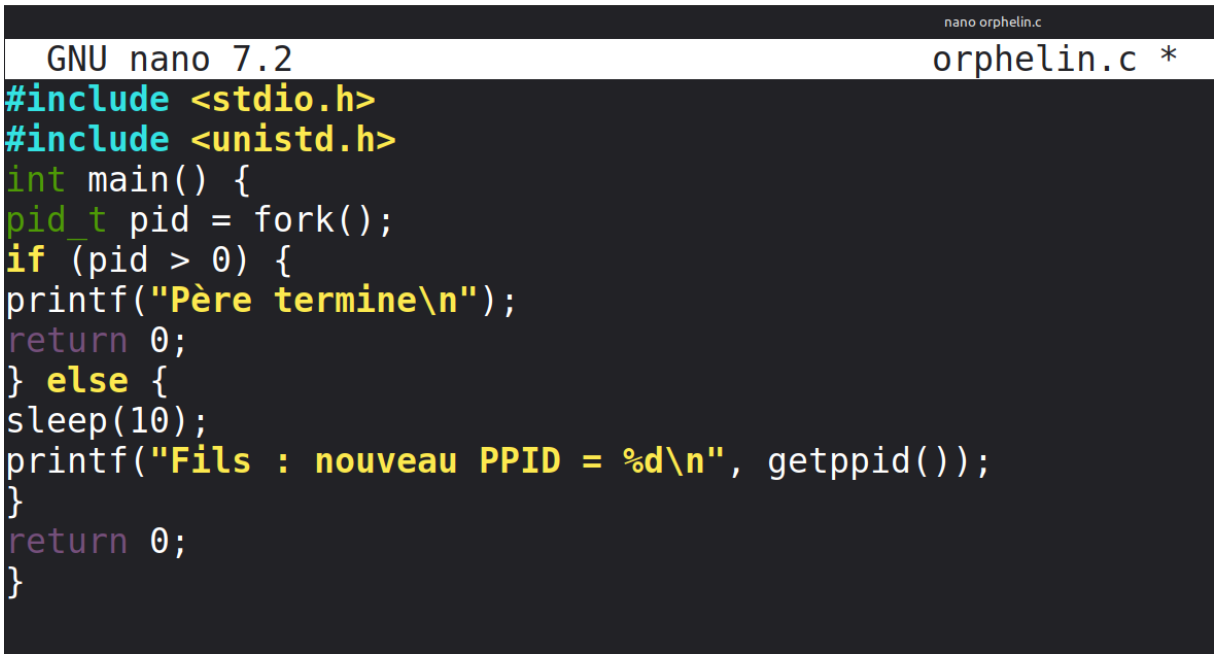
3 - the child process (PID 45715) shows the state Z.

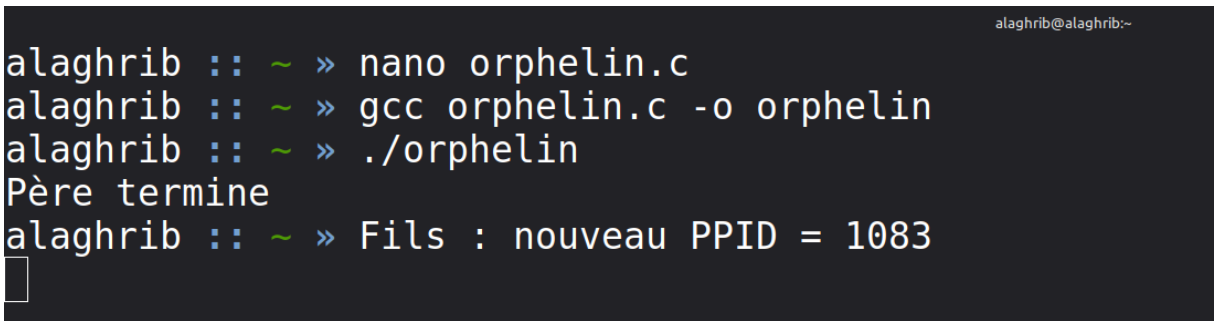
4 - t stands for Zombie (or defunct). It indicates a process that has completed execution but still has an entry in the process table.

5 - A process becomes a zombie because its parent has not yet read its exit status via the wait() system call.

6 - By ensuring the parent process calls wait() or waitpid() to collect the termination status of its children.

Part 05 : Orphan Process





Questions :

1 - Two processes : the parent and one child process.
2 - The child process continues execution in the background while the parent finishes and disappears from the process list.
3 - It becomes an Orphan process.
4 - The new PPID is typically 1083.
5 - To ensure that every process has a parent to collect its exit status and prevent it from remaining in the system indefinitely.

Part 06 : Replacement with exec()

```
alaghrib@alaghrib:~
alaghrib :: ~ » nano exec1.c
alaghrib :: ~ » gcc exec1.c -o exec1
alaghrib :: ~ » ./exec1
Avant exec
total 176
drwxrwxr-x 3 alaghrib alaghrib  4096 Mar 14 07:23 Archi
drwxrwxr-x 3 alaghrib alaghrib  4096 Mar 14 07:32 computer-architecture
drwxr-xr-x 2 alaghrib alaghrib  4096 Mar 31 12:07 Desktop
drwxr-xr-x 5 alaghrib alaghrib  4096 Mar 14 08:17 Documents
drwxr-xr-x 4 alaghrib alaghrib  4096 Mar 31 12:07 Downloads
-rwxrwxr-x 1 alaghrib alaghrib 16000 Mar 31 17:33 exec1
-rw-rw-r-- 1 alaghrib alaghrib   169 Mar 31 17:31 exec1.c
-rwxrwxr-x 1 alaghrib alaghrib 16128 Mar 31 12:12 fork1
-rw-rw-r-- 1 alaghrib alaghrib   366 Mar 31 12:12 fork1.c
-rwxrwxr-x 1 alaghrib alaghrib 16088 Mar 31 14:56 fork2
-rw-rw-r-- 1 alaghrib alaghrib   157 Mar 31 14:55 fork2.c
-rwxrwxr-x 1 alaghrib alaghrib 16088 Mar 31 14:22 fork_wait
-rw-rw-r-- 1 alaghrib alaghrib   266 Mar 31 14:22 fork_wait.c
drwxr-xr-x 2 alaghrib alaghrib  4096 Feb  7 16:26 Music
drwxrwxr-x 3 alaghrib alaghrib  4096 Mar 14 17:48 oop
drwxrwxr-x 3 alaghrib alaghrib  4096 Mar 14 07:32 operating-systems
-rwxrwxr-x 1 alaghrib alaghrib 16136 Mar 31 17:23 orphelin
-rw-rw-r-- 1 alaghrib alaghrib   207 Mar 31 17:23 orphelin.c
drwxr-xr-x 2 alaghrib alaghrib  4096 Mar 31 17:31 Pictures
drwxrwxr-x 6 alaghrib alaghrib  4096 Mar 14 11:35 pt
drwxr-xr-x 2 alaghrib alaghrib  4096 Feb  7 16:26 Public
drwxr-xr-x 2 alaghrib alaghrib  4096 Feb  7 16:26 Templates
drwxr-xr-x 2 alaghrib alaghrib  4096 Feb  7 16:26 Videos
drwxrwxr-x 4 alaghrib alaghrib  4096 Mar 14 08:14 web-development
-rwxrwxr-x 1 alaghrib alaghrib 16048 Mar 31 17:03 zombie
-rw-rw-r-- 1 alaghrib alaghrib   190 Mar 31 17:01 zombie.c
```

```
nano exec1.c
  GNU nano 7.2                                  exec1.c *
#include <stdio.h>
#include <unistd.h>
int main() {
printf("Avant exec\n");
execl("/bin/ls", "ls", "-l", NULL);
printf("Après exec\n"); // Ne s'affiche pas
return 0;
}
```

Questions :

1 - Because exec() replaces the entire address space of the current process with the new program. The code following the exec() call is no longer in memory.

2 - it replaces the current running program with a new one while keeping the same PID and PPID.

3 - fork() creates a new duplicate process, while exec() replaces the existing process with a different program .

Part 07 : fork() + exec() Combined

```
  GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
pid_t pid = fork();
if (pid == 0) {
execl("/bin/date", "date", NULL);
} else {
wait(NULL);
printf("Commande terminée\n");
}
return 0;
}
```

```
alaghrib :: ~ » nano fork_exec.c
alaghrib :: ~ » gcc fork_exec.c -o fork_exec
alaghrib :: ~ » ./fork_exec
Tue Mar 31 06:06:03 PM CET 2026
Commande terminée
```

Questions :
1 - Two processes : the parent and the child.
2 - The child process executes the date command using the execl() system call.
3 - wait() is used to synchronize the two processes ; it forces the parent to wait until the child finishes executing the date command before printing "Commande terminée".